

MEDIROZA GENERAL HOSPITAL

INDEPENDENT WEB APPLICATION SECURITY ASSESSMENT

PORTFOLIO-SAFE CASE STUDY

Prepared by Chandrashekar Bala

Authorized black-box assessment • 01 October 2026 • Public-safe edition

This version is sanitized for public portfolio use. Credentials, patient records, national identifiers, direct contact data and raw sensitive artifacts are intentionally excluded.

Project Overview

This standalone case study demonstrates an end-to-end web application security assessment performed from an external, unauthenticated position under written authorization. The work combines reconnaissance, manual exploitation, evidence validation, document-security analysis, impact quantification, remediation engineering and retest design.

Capability	Demonstrated work
Web reconnaissance	DNS, HTTP metadata, robots/sitemap, application mapping
Offensive testing	Authentication testing, SQL injection, directory exposure, object-access analysis
Tooling	Nmap, curl, Burp Suite, sqlmap, qpdf, pdfcrack
Evidence engineering	Baseline/exploit comparison, screenshots, hashes, command register
Defensive engineering	Root-cause analysis, remediation patterns, detection rules, regression tests

Executive Findings

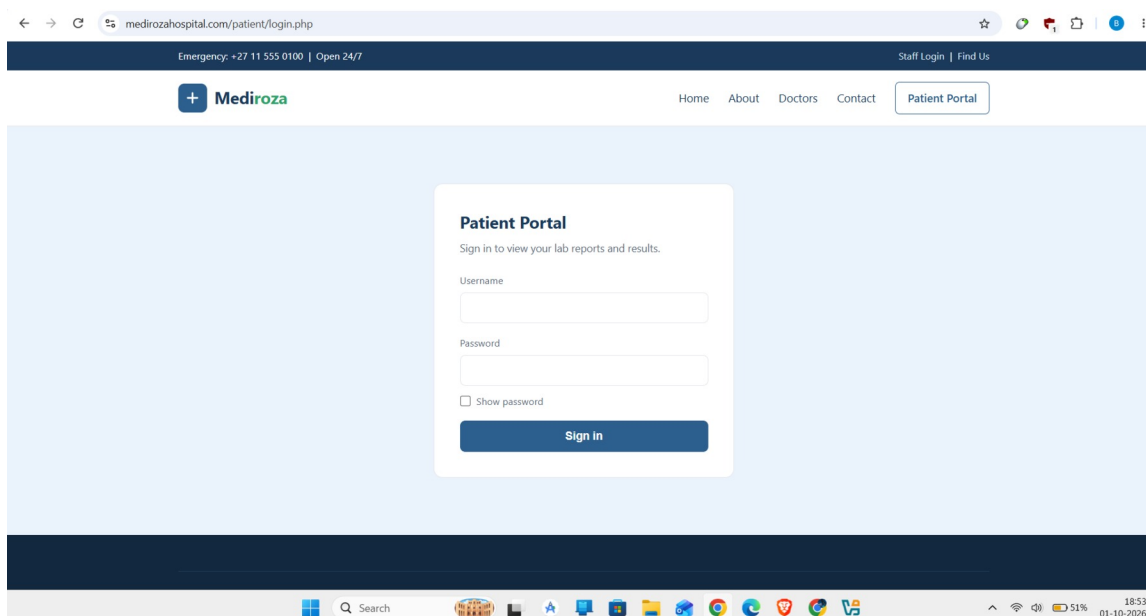
ID	Finding	Severity	CVSS
F-01	Unauthenticated directory indexing	HIGH	7.5
F-02	Public internal database backup exposure	CRITICAL	9.4*
F-03	SQL injection authentication bypass	CRITICAL	9.8
F-04	Verbose SQL error disclosure	MEDIUM	5.3
F-05	Sensitive report retrieval after auth bypass	HIGH	8.1*
F-06	Predictable/disclosed report path	LOW	3.1*
F-07	Error-log metadata disclosure	MEDIUM	5.3*
F-08	Server/hosting fingerprint disclosure	LOW	3.1*

The most important technical result is the independently reproducible authentication-state change: a normal invalid login returned HTTP 200 with no redirect, while a controlled SQL syntax mutation returned HTTP 302 to the patient portal. Browser evidence confirmed the resulting portal state.

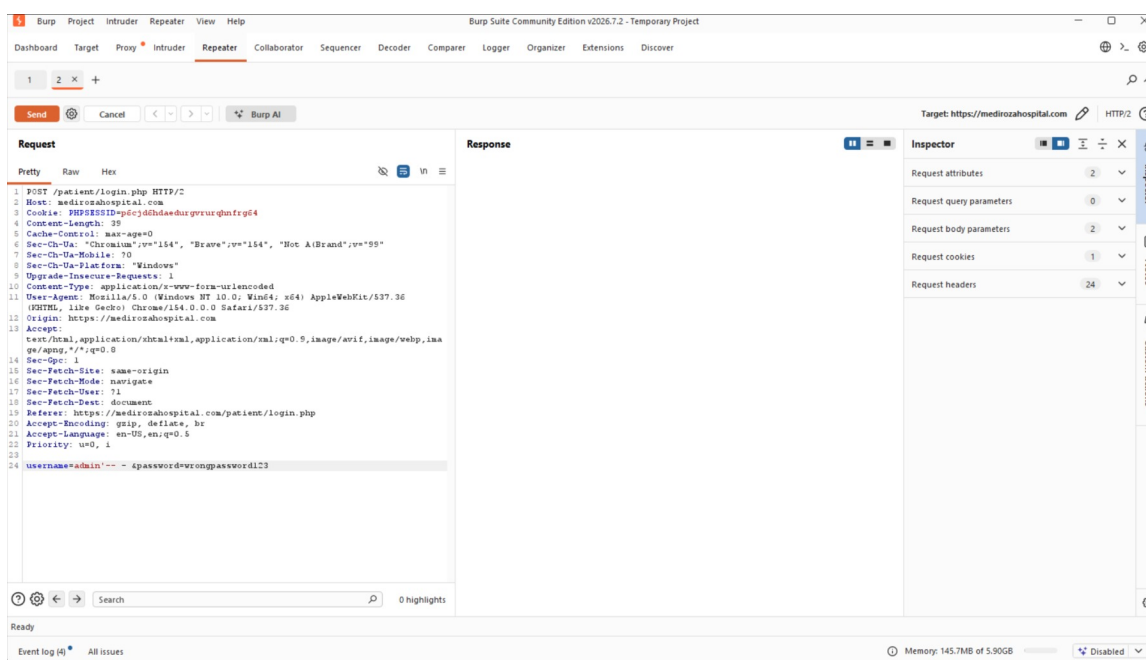
Technical Deep Dive — SQL Injection

The login endpoint accepted username and password via POST. The baseline request established normal behavior. The exploit request changed only the username field. The application then redirected to the patient portal. The exact server-side source code was not available, so the report describes the root cause at the observable control level: untrusted input is influencing database query behavior.

A secure implementation should use parameterized queries so the database receives the username as data rather than executable SQL syntax. Production error handling should also suppress database-driver messages and preserve diagnostics only in protected server-side logs.



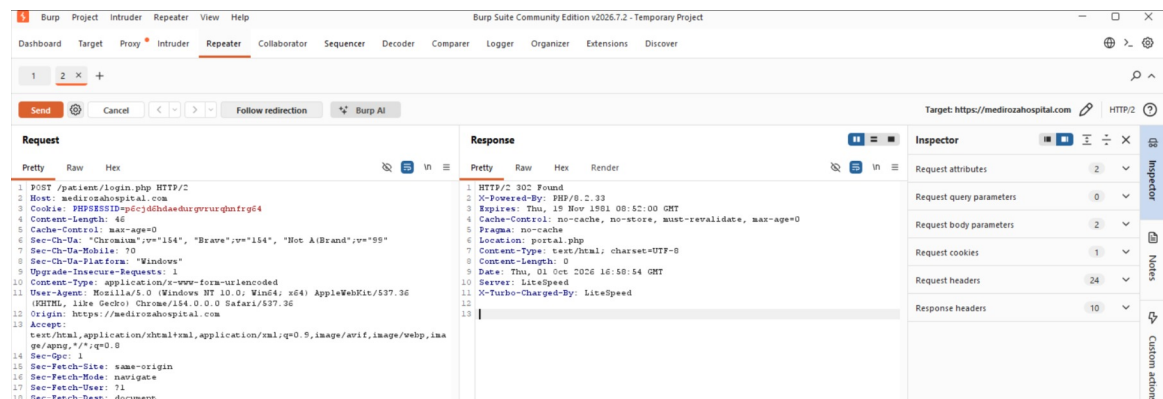
Patient login surface — public-safe evidence.



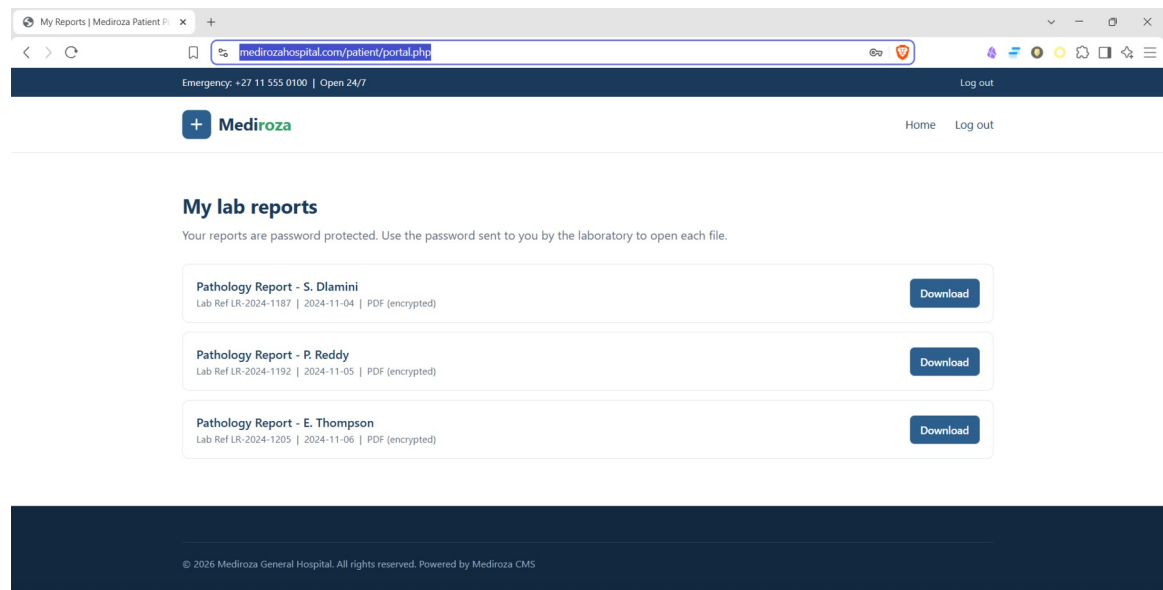
Controlled SQL injection test — public-safe evidence.

Evidence Quality & Reproducibility

- A baseline request is captured before the exploit request.
- The exploit changes one material input rather than several variables at once.
- Burp/browser evidence corroborates the HTTP response.
- sqlmap limitations are reported rather than overstated.
- Impact is demonstrated with actual HTTP responses and retrieved object types.
- The confidential evidence package is indexed and integrity-checked separately.



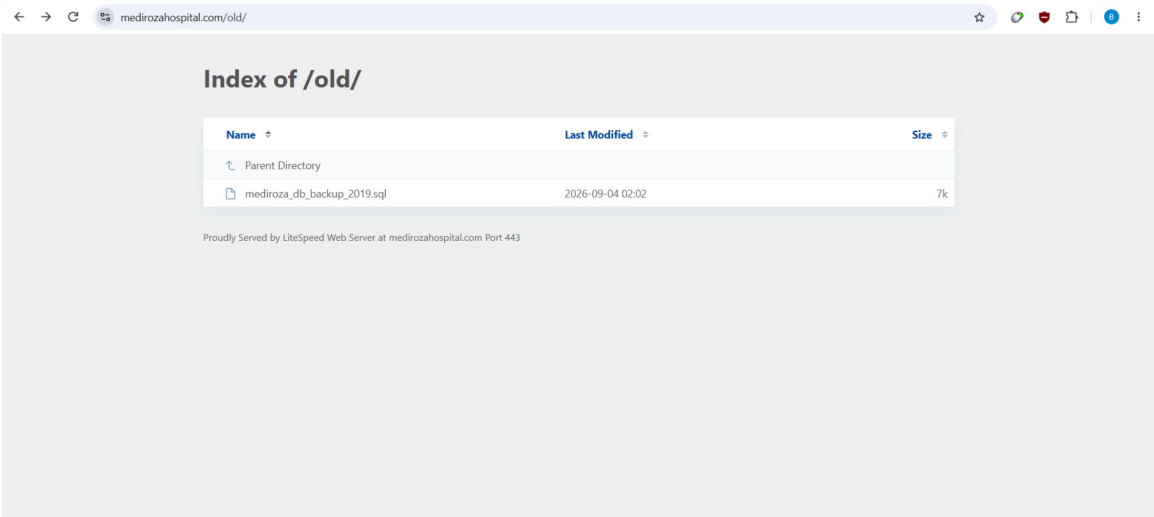
HTTP 302 authentication-bypass response.



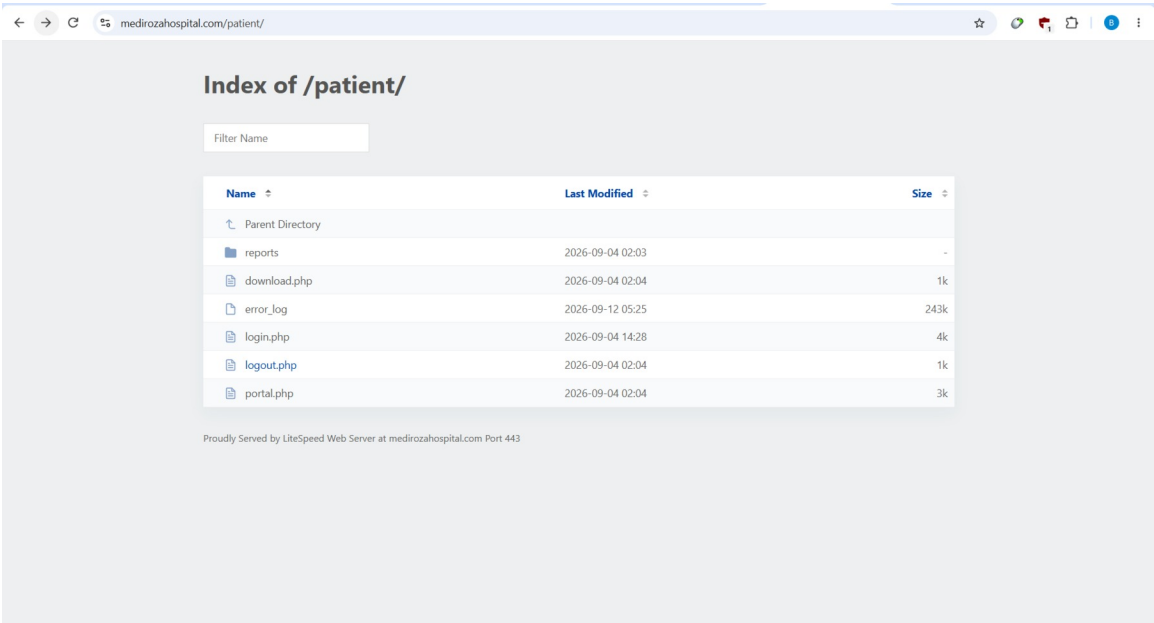
Browser-confirmed portal access.

Directory & Backup Exposure

Directory indexing exposed application structure under restricted paths. The most consequential result was discovery of a database backup in a legacy directory. The public-safe version does not reproduce the sensitive database contents; the confidential edition contains the complete technical record.



Legacy directory listing — sensitive filenames redacted in public use.



Patient directory enumeration evidence.

Three encrypted PDF artifacts were retrieved after the authentication boundary was bypassed. The assessment recorded PDF 1.4, Standard security-handler parameters $V=2/R=3$ and 128-bit protection. Authorized offline password recovery succeeded against all three files. Passwords and report contents are deliberately excluded from this public-safe version.

PDF hash extraction evidence — credentials removed from public release.

Authorized document-security testing — public-safe context.

Impact Quantification

The confidential database backup contained structured staff and shareholder datasets. The technical analysis established 30 staff records and 10 shareholder records and calculated an exact monthly payroll total of ZAR 2,027,000. The public version intentionally omits individual identities, national identifiers, contact details, salary rows and ownership rows.

Data class	Observed quantity
Staff records	30
Shareholder records	10
Monthly payroll total	ZAR 2,027,000
Patient report artifacts	3 retrieved PDFs

Root Causes

- Sensitive artifacts were placed under the public web document root.
- Directory indexing disclosed files that should have been private.
- Authentication input was not safely isolated from database query semantics.
- Sensitive report objects were reachable through predictable identifiers after the auth boundary was bypassed.
- Production errors disclosed database implementation details.

Remediation Architecture

Control	Engineering action
Public-file hygiene	Move backups/logs/archives outside the web root
Authentication	Use prepared statements and least-privileged DB access
Object authorization	Authorize every document request against session identity and record ownership
Error handling	Return generic client errors; log details server-side
Monitoring	Correlate failed login sequences with portal-session creation
CI/CD	Block database dumps and backup artifacts from deployment

Retesting should verify the security invariant rather than only one payload. Directory indexes should be gone; known backup names should return 403/404; SQL-like inputs should never change authentication state; unauthorized report IDs should return a consistent denial; and the expected security telemetry should be visible to defenders.

Resume-ready Project Description

Independent Web Application Security Assessment — Conducted an authorized black-box assessment of a hospital web application; identified and manually validated SQL-injection authentication bypass, public database-backup exposure, directory indexing and sensitive document retrieval; analyzed legacy PDF protection; quantified exposed corporate data; and produced evidence-backed severity, remediation, detection and retest artifacts.

Public Release Boundary

Use this sanitized edition for a portfolio, GitHub repository or resume-linked project page. Keep the confidential technical report and evidence package private because they contain recovered credentials and sensitive personal/corporate data. The public version preserves the technical story without exposing those records.